DeepSonar

One-view 3D object completion from a low-resolution time-of-flight sensor

Complete project report: data, preprocessing, model selection, training, evaluation, deployment and limits

Prepared for faculty evaluation, the exhibition of 3 November 2026, and paper preparation

The result in one paragraph. From a single hand-held scan of one side of an object, the system predicts the complete object as 16,384 points and a closed 3D surface, in metres, running on the CPU of the viewing device. An official pretrained AdaPoinTr network was fine-tuned on about 66,000 device-faithful simulated scans and on 18 real sessions. On two held-out real sessions of the same two objects, the output is 4.8 mm (bottle) and 3.1 mm (box) from a reference surface built from the known object. The hidden side is predicted, never measured, and the real-scan performance is specific to these two objects.

Final model: run final7, weights selected at step 138,000. Report generated from the project logs. 1 November 2026.

Contents

1. Summary **4**

What was built . . . 4

Main findings . . . 4

2. Hardware and acquisition **5**

How a scan is made . . . 5

The real data . . . 5

3. Problem statement **6**

4. Model selection **7**

AdaPoinTr in brief . . . 7

5. Data **8**

Why simulation . . . 8

The simulator . . . 8

The dataset . . . 8

Augmentation . . . 8

6. Preprocessing **9**

Thinning . . . 9

Cleaning . . . 9

Canonical frame . . . 9

Sampling . . . 9

7. Training **10**

Hyperparameters . . . 10

How the settings were chosen . . . 10

Every training run . . . 10

Full targets for real scans . . . 10

8. Resources and engineering **12**

9. Evaluation **13**

Simulated test split . . . 13

Real scans . . . 14

How results changed by stage . . . 15

10. From points to a 3D object **16**

11. A numerical finding: 16-bit ranking **17**

The symptom . . . 17

The cause . . . 17

The fix . . . 17

12. The web application **18**

Native apps . . . 18

13. Limits and threats to validity **19**

14. Project journey **20**

Lessons 20

15. Exhibition guide **21**

Questions to expect 21

16. Towards a paper **22**

Possible contributions 22

What a paper still needs 22

17. Reproducibility **23**

References 23

1. Summary

A depth sensor sees only the side of an object that faces it. Shape completion asks a model to supply the rest. This project applies that idea to DeepSonar, a low-cost hand-held device built from an ESP32, an ST VL53L5CX 8 by 8 time-of-flight sensor and a BNO08X motion sensor.

The hard part is the data, not the network. Published completion models expect clean, dense partial scans. DeepSonar gives a few hundred smeared points, taken with an object that covers only about two of the sensor's eight zones, with hand motion that the device does not track, mixed with the ceiling, table and room.

What was built

- A cleaner that finds the scanned object in a raw file using geometry alone, with no labels.
- A simulator that reproduces the real sensor and the real scan sessions, used to generate about 66,000 training pairs.
- Fine-tuning of the official AdaPoinTr weights, in stages, on simulated pairs and on real scans.
- Reference targets for real scans, built by registering the known object to each scan's front face.
- A surface step that turns the output points into one closed 3D object.
- A web application in which the model runs in the browser, on the device's own CPU.

Main findings

- Training on simulation alone gave correct heights but fuzzy, inconsistent shapes on real scans.
- Adding real scans with a front-only loss taught the model to copy the smear. It improved fit to the input and did not improve the shape.
- Adding full targets for the known objects removed the smear: bottle widths from 4.6 to 10.7 cm in the input became a consistent 7 cm on sessions that had targets.
- A numerical bug, explained in Section 11, made results 2 to 3 times worse on CPU and in the browser than on the training GPU. It was found and fixed without retraining.
- The cost of known-object targets is measurable. When a box input is narrowed to half its width, the final model's output barely changes (ratio 0.88, where 0.5 is full tracking and 1 is a fixed shape). The first model scored 0.48. The final model is strongly biased toward the two objects it learned.

What must not be claimed. The back of a real object was never measured. The real-scan targets are the known object placed on the scan, not a measurement. The held-out sessions are new scans of the same bottle and box, so they say nothing about other objects.

2. Hardware and acquisition

Part	Details
Controller	ESP32
Depth sensor	ST VL53L5CX, 8 x 8 zones, about 45 x 45 degree field of view, about 5.6 degrees per zone, 15 Hz
Motion sensor	BNO08X, game rotation vector
Camera	On the device; 8 unpaired, blurry frames were available and were not used for training
Output	A raw metric map as a NumPy .npy file: N x 3 points in metres, z up, sensor at the origin

Table 1. The device.

How a scan is made

The device lies on the table facing up, is picked up, is aimed at the object for about 25 seconds, and is put down. The viewer turns each frame into 3D points and registers them using rotation only. Translation is not tracked, so hand drift stays in the file as smear. One file therefore mixes the ceiling, swing streaks, table, background furniture, near-sensor junk, and the object.

The real data

	Box	Bottle
Files	14	13
Sessions, after merging repeat exports	11	9
Typical distance	26 to 39 cm	30 to 37 cm
Points on the object after cleaning, median	1,058	557
Depth drift during the hold, median	4.6 cm	5.6 cm
True size	Tall, thin box: about 18.8 x 23.4 cm from the scans; depth given by the owner only as very small; 2 cm assumed	Red flip-cap bottle, about 7 cm wide, 25 cm tall (photo estimate, not a ruler measurement)

Table 2. Real data. Sessions 121600 (box) and 123815 (bottle) were held out before any result was seen.

3. Problem statement

Input: one raw scan file from one viewpoint. **Output:** the complete object in metres as a point cloud and a 3D surface. **Constraints:** one file per object, no labels at inference, on-device inference, an open-source pretrained model, and honest reporting of what is predicted.

The task has a limit that no training can remove. One face of a box does not contain its depth. A network can only learn a prior from training examples. A round object is different, because the curve of the front suggests the radius.

4. Model selection

Option	Decision	Reason
AdaPoinTr, official PCN weights	Selected	Strong published results, MIT licence, official weights, output of 16,384 points, and small enough to run in a browser
PCN	Used as baseline only	The evaluation compares against the official AdaPoinTr PCN weights and against returning the input
Train from scratch	Rejected	The project requires an official pretrained model, and fine-tuning needs far less compute
Class label followed by a stored shape	Rejected	That would be a lookup, not completion. An early version was rejected for this reason. The network input is 2048 points only
Pixal3D (image to 3D)	Not used	Invents the back, has no metric scale, needs a large GPU, and does not use the sensor
Photo description plus sensor scale (hybrid)	Tested, not shipped	A short trial fitted a vision model's shape to the scan. Sizes were right; the median fit error was 9.0 to 9.5 mm against an 8 mm limit. It depends on a cloud service
Forcing the points to a template in preprocessing	Rejected	It would make the model pointless and would hide what the network does

Table 3. Options considered.

AdaPoinTr in brief

AdaPoinTr (Yu et al., 2023) groups the input points, encodes them with geometry-aware transformer blocks, predicts a coarse set of centres, selects the best query points, decodes them, and rebuilds local point patches around each centre. The configuration used is the official PCN one: 2048 input points, 16,384 output points, 512 queries, embedding size 384, and about 32.5 million parameters. The official code was ported into this repository with small shims for CPU, GPU and ONNX.

5. Data

Why simulation

Twenty real sessions cannot train a network, and no real scan has a measured back. Training pairs must be simulated, so the simulator has to reproduce the device. The rule set by the project owner was that simulated samples must follow the real environment.

The simulator

The simulator models the sensor's zone geometry, noise that grows with range, zone-specific validity, the frame timeline and the rotation-only registration. Two regimes were used.

Regime	What it does	Share of training data
real	Replays measured frames from the 18 training sessions, cuts out the real object, and ray-casts a new object at the session's measured place and drift	about 92%
robust	Invented rooms with wider ranges, for robustness	about 8%

Table 4. Simulator regimes.

Every simulated scan then passes through exactly the deployment pipeline. Scans the cleaner rejects are skipped and counted, so training sees what deployment sees.

The dataset

Split	Scans kept	Notes
Train	64,847 generated; 61,137 used	Scans with under half of the cleaned points on the object are dropped from training
Validation	3,675	Selects the checkpoint, on the real regime only
Test	1,857	9.3% of drawn test scans were rejected by the cleaner

Table 5. Dataset, ten object kinds: cuboid, cube, slab, tower, bottle, wavy bottle, can, jar, rod, pipe.

Each pair is 2048 input points and a 16,384-point target. Each shard records code, configuration and dataset hashes. One corrupted shard was found through a checksum failure during a transfer, re-fetched and re-verified.

Augmentation

Mirror, yaw of 10 degrees and scale from 0.95 to 1.05 are applied to input and target together. Dropout, half-plane crops, local holes, depth copies of 1 to 6 cm and jitter are applied to the input only.

6. Preprocessing

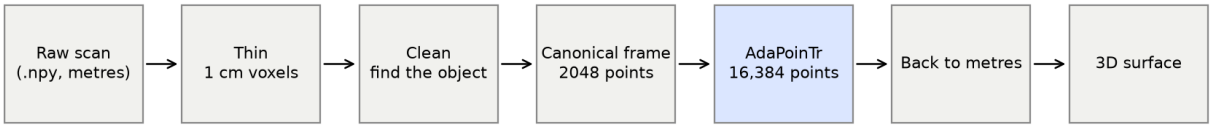


Figure 1. The deployment pipeline. The same code path is used for training data, evaluation and the applications.

Thinning

Raw maps are reduced to one point per 1 cm voxel, matching what the device viewer stores.

Cleaning

The cleaner defines the signal, not the noise. The object is the surface the user aimed at for seconds, so it is dense, locally a smooth surface, compact, upright, and clearly stronger than any other candidate. Thresholds come from the file's own point spacing and from sensor physics. There are no labels and no learned parts. If two candidates are comparable the file is rejected instead of guessed. Hand drift is handled by keeping the dominant depth layer. All 20 real sessions are accepted.

Known limit. Only upright targets are accepted. A flat object lying on a table is treated as table.

Canonical frame

Points are rotated about the vertical so that +z points from sensor to object, centred on the seen face, and scaled so that the longest seen side is 0.9 units. This keeps metric scale recoverable: the transform is stored and inverted on the output.

Sampling

Farthest-point sampling gives exactly 2048 points. It is deterministic, so Python and JavaScript produce bit-identical inputs. All 42 test files match exactly.

7. Training

Hyperparameters

Parameter	Value	Note
Optimiser	AdamW	Weight decay 5e-4
Learning rate	1e-4 (stage 1, branches); 5e-5 (final stages)	Cosine decay to 2% with 500 warm-up steps
Batch	16 simulated, plus 8 real in stage 2	
Precision	Automatic mixed precision, float16	See Section 11
Gradient clipping	10.0	
Model averaging	EMA, decay 0.999	EMA weights are shipped
Evaluation	Every 2,000 steps	Best checkpoint on real-regime validation
Real loss weight	3.0	0.5 originally planned; raised to emphasise real scans
Loss	AdaPoinTr denoise and reconstruction Chamfer L1	Plus real-scan loss

Table 6. Main training settings.

How the settings were chosen

There was no grid search: each run costs real credits, and the budget allowed only a handful. Settings began from the official configuration. Each later change answered a specific measured problem and was compared on the same held-out sessions. Two recipes were run side by side from the same checkpoint so that they could be compared fairly.

Every training run

Run	What	Steps	Epochs	Minutes	Peak LR	Finished
final1	Stage 1: simulated scans only	0 to 37,400	9.79	145.8	0.0001	yes
final2	Real scans, front-only loss. Died when a local process ended	37,446 to 57,800	5.33	75.8	0.0001	stopped
final3	Branch A: same recipe, continued. Stopped: it copied the smear	56,000 to 84,000	7.33	110.6	0.0001	stopped
final4	Branch B: full bottle targets	56,000 to 128,200	18.9	295.9	0.0001	yes
final5	Box focus: full box targets added	94,000 to 107,600	3.56	63.1	5e-05	stopped
final7	Final: 10 epochs, plain mix, full targets	104,000 to 142,100	9.97	180.9	5e-05	yes
final6	Plain mix. Stopped by the owner	107,639 to 125,500	4.67	78.1	5e-05	stopped

Table 7. All runs. One epoch is 3,821 steps. The final model's lineage is final1, final2, final4, final5, final7, about 36 epochs up to the selected step.

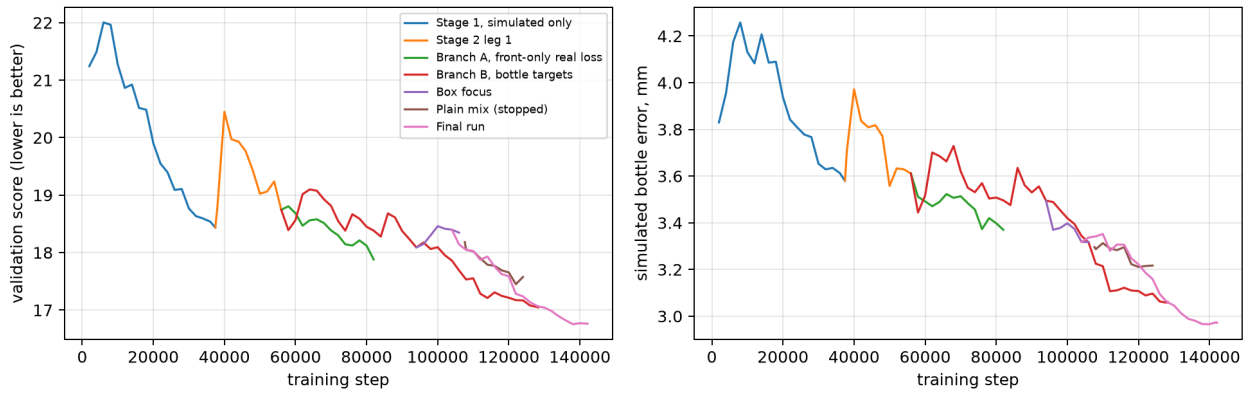


Figure 2. Simulated validation for every run. A jump at the start of a run comes from the new learning rate and loss.

Full targets for real scans

A front-only loss cannot teach the back. The object owner proposed building a target from the known object. For a real scan, the known surface is scaled to the object's height and fitted to the cleaned

front face, by translation and rotation about the vertical only. A target is accepted when the median distance from the face to the surface is at most 12 mm and its sensor-facing side lies inside the face's depth range. Accepted sessions train with the same two-sided loss as simulated pairs.

Object	Source of shape	Accepted	Fit error
Bottle	Outline read from a photo by a vision model	7 of 9 sessions (123028 and 123523 rejected)	8.7 to 11.0 mm median
Box	Cuboid 18.8 x 23.4 x 2 cm. The depth is an assumption	11 of 11 sessions	4.1 to 6.7 mm median

Table 8. Real-scan targets.

Disclosure. These targets are the known object registered to the scan. They give the network the object's true form, so the network learns these two objects. The real-scan result is therefore instance-specific.

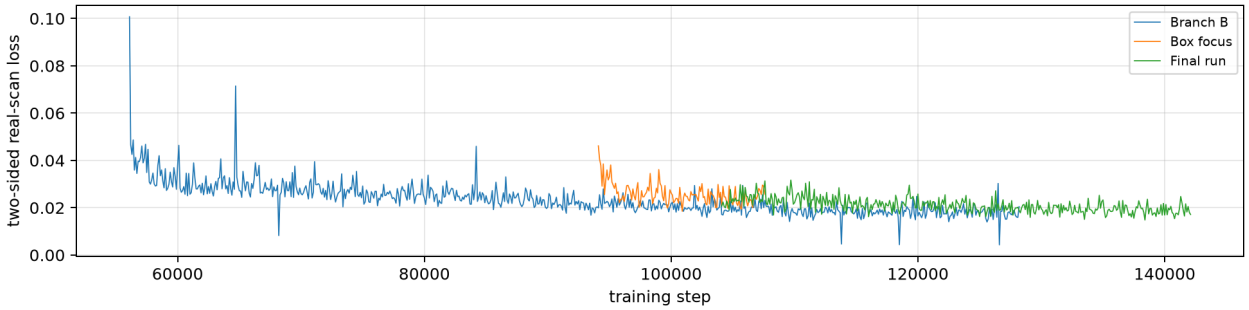


Figure 3. Real-scan loss with full targets.

8. Resources and engineering

Resource	Details
Training compute	Modal cloud, one NVIDIA H100, 8 CPU cores, 32 GB memory, about $4.58 per hour
Spend	About $84 of GPU time in total
Local machine	Laptop CPU for evaluation, export, tests and app building
Software	Python 3.11, PyTorch, NumPy, SciPy, scikit-image, ONNX Runtime Web, Plotly, uv
Code size	About 12,300 lines of Python across the package, scripts and tests
Tests	179 automated tests in 10 test files, plus a Python to JavaScript parity test on 42 files

Table 9. Resources. Spend comes from Modal billing reports; the most recent charges can lag.

A spending guard refuses any launch whose worst case would cross the cap. Every function has a timeout, no retries and one container. Each run logs every 100 steps and validates every 2,000 steps. Each checkpoint stores code, configuration and dataset hashes.

9. Evaluation

Eighteen pass or fail gates were written before the first result was seen and were not changed afterwards. Final model: **10 of 18 pass**. Metric: Chamfer L1, the mean nearest-point distance in both directions, averaged, in millimetres.

gate	result	detail
sim: median CD below returning the input, every kind	PASS	failing kinds: none
sim: median CD below official PCN weights, every kind	PASS	failing kinds: none
sim: median CD at most 5.0 mm, every kind	FAIL	failing kinds: cuboid, cube, slab, tower
sim: mean F@10 mm at least 0.8, every kind	FAIL	failing kinds: cuboid, cube
real: every one of the 20 sessions gives an output	PASS	rejected: none
real: one-sided median at most 10.0 mm	FAIL	2 sessions fail: 122622 123146
bottle: section residual / radius at most 0.1	FAIL	2 sessions fail: 123028 123523
bottle: angular coverage at least 0.9	FAIL	2 sessions fail: 123028 123523
bottle: section aspect at least 0.8	FAIL	2 sessions fail: 123028 123523
box: median face flatness at most 5.0 mm	PASS	all sessions pass
box: worst right-angle error at most 5.0 deg	PASS	all sessions pass
box: at least 0.8 of the output within 8 mm of a fitted face	PASS	all sessions pass
box: width std/mean across sessions at most 0.1	PASS	0.018 over 11 sessions
box: height std/mean across sessions at most 0.1	PASS	0.010 over 11 sessions
bottle: width std/mean across sessions at most 0.1	PASS	0.099 over 9 sessions
bottle: height std/mean across sessions at most 0.1	PASS	0.024 over 9 sessions
probes: plate, rod, cube, noise differ, margin at least 3.0	FAIL	margin 0.92
probe: width/height of the output follows a halved width, ratio at most 0.75	FAIL	ratio 0.882; 0.5 is full tracking, 1 is a fixed shape

Table 10. Gates for the final model. A star marks a held-out session.

Stage 1 passed 7 of 18. The gain to 10 comes from the box shape gates, and the bottle gates now fail only on the two sessions without targets. Two probe gates that stage 1 or the pilot passed now fail, which is the signature of specialising to the two objects.

Most important negative result. The input-sensitivity probes fail. The ratio moved from 0.48 in stage 1 to 0.88: a narrowed box still gives close to the learned box. The model remains a network that reads its input, with no label or lookup, but on box-like inputs it behaves close to a learned template. This is the expected price of fine-tuning on one instance, and it must be stated when the real-scan numbers are quoted.

Simulated test split

group	n	CD median mm	official PCN median	input median	F@10 mm	official F@10	W err mm	H err mm	D err mm
cuboid	296	6.5	30.9	31.7	0.782	0.312	19.3	4.7	62.4
cube	97	5.7	50.9	47.3	0.789	0.230	21.9	3.7	33.6
slab	77	6.8	14.4	14.1	0.830	0.452	15.6	4.2	70.7
tower	102	5.3	15.2	17.8	0.907	0.440	6.3	6.5	30.9
bottle	371	2.9	13.6	15.8	0.991	0.464	3.2	5.1	3.3
wavy_bottle	220	2.9	13.4	15.5	0.990	0.475	3.8	4.1	3.7
can	124	2.7	17.4	18.4	0.985	0.422	4.4	3.7	4.6
jar	129	3.1	22.4	22.8	0.979	0.365	5.2	3.1	5.2
rod	100	3.2	9.1	11.4	0.984	0.579	4.3	9.7	6.2
pipe	121	3.8	19.1	22.0	0.961	0.444	5.0	8.4	4.9

Table 11. Simulated test, real regime, cleaner on the object. The trained model beats the official weights and the input on every kind.

The errors in W, H and D are width, height and depth. Cuboid and slab depth errors remain large: the depth of a simulated box with unknown proportions cannot be read from one face.

Real scans

Real scans have no measured back, so three separate checks are reported: consistency, distance to the reference surface, and fit to the measured front.

Check	Bottle	Box
Held-out session, distance to reference, mm	4.79	3.12
Training sessions with targets, median, mm	1.02	1.28
Width variation across sessions (std / mean)	0.099	0.018
Median width, cm	7.1	18.3
Median height, cm	23.0	23.5

Table 12. Real scans, final model, CPU at browser precision. The gap between held-out and training sessions shows that training sessions are partly memorised.

Session	Object	Used in training	Distance to reference, mm	Width, cm	Depth along view, cm	Height, cm
122622	Bottle	yes	1.01	7.1	7.1	22.8
122735	Bottle	yes	1.09	7.2	7.2	23.0
122907	Bottle	yes	0.92	6.9	6.9	22.1
123028	Bottle	yes	-	6.8	9.1	24.3
123146	Bottle	yes	1.04	7.2	7.2	23.1
123259	Bottle	yes	1.03	7.1	7.1	22.9
123523	Bottle	yes	-	5.0	7.8	23.0
123658	Bottle	yes	0.98	7.3	7.3	23.3
123815	Bottle	held out	4.79	6.9	6.9	22.6
114126	Box	yes	1.33	18.5	6.3	23.5
115102	Box	yes	1.25	18.6	5.2	23.4
115330	Box	yes	1.25	18.3	6.9	23.4
115840	Box	yes	1.21	18.7	5.2	23.5
120051	Box	yes	1.30	18.4	6.4	23.5
120408	Box	yes	1.32	18.3	6.9	23.4
120745	Box	yes	1.28	18.1	7.3	23.4
121055	Box	yes	1.21	18.2	7.7	23.5
121223	Box	yes	1.28	17.6	9.1	23.5
121457	Box	yes	1.27	18.3	7.0	23.5
121600	Box	held out	3.12	18.9	6.8	24.3

Table 13. All 20 real sessions. Two bottle sessions have no reference because their target failed the fit check. Box depth along the view includes the box's turn of about 15 degrees to the sensor, so it is larger than its thickness.

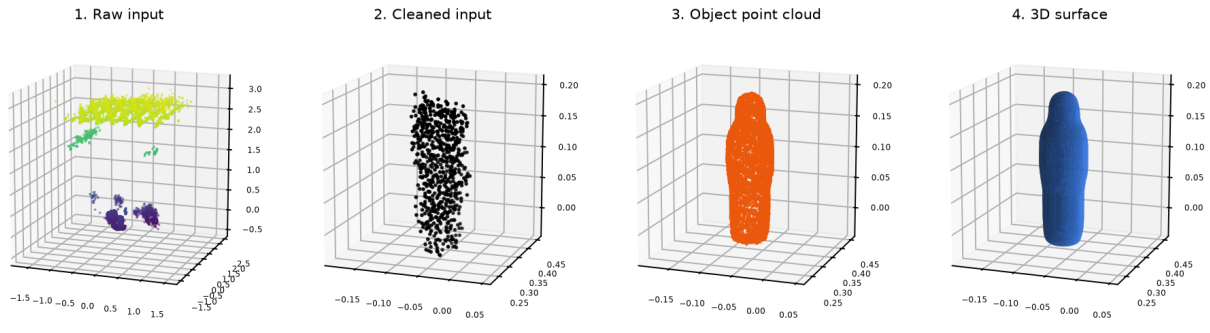


Figure 4. Held-out bottle session 123815. The cleaned input is only the front.

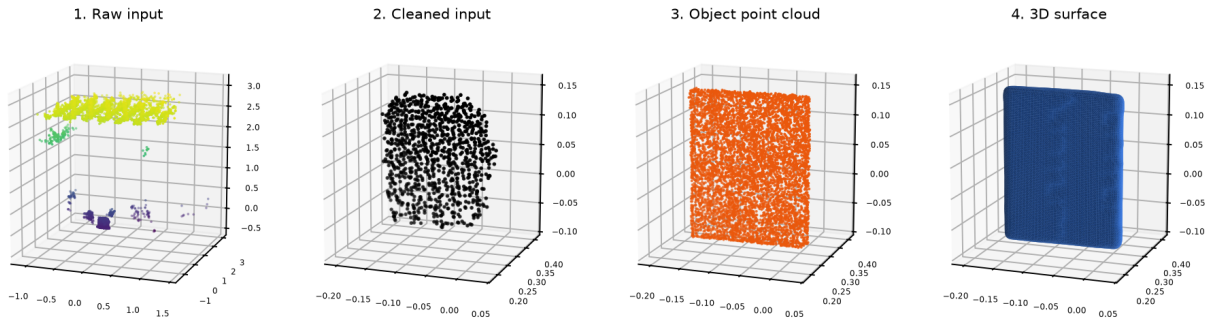


Figure 5. Held-out box session 121600.

How results changed by stage

Model	Bottle held-out, mm	Box held-out, mm	Bottle width variation	Box width variation
Stage 1, simulated only	16.81	8.24	0.238	0.153
Branch A, front-only real loss	8.55	9.47	0.260	0.101
Branch B, bottle targets	6.35	9.92	0.085	0.106
Final, with the ranking fix	4.79	3.12	0.099	0.018

Table 14. Stage comparison. The first three rows were measured before the fix in Section 11, so they understate those models. Box distances use the assumed 2 cm depth, which earlier models never trained on.

10. From points to a 3D object

The surface step wraps the predicted points in one closed surface. Stray points are dropped. Each horizontal slice is wrapped with its convex outline, and the outlines are joined and smoothed. It uses no template and no object label, so its shape comes from the model.

An earlier attempt that filled a voxel grid failed because noisy thin shells leak, and it broke into pieces. Slice-wise filling solved this. It suits upright objects with convex cross-sections; a concave section would be filled in. The laptop version uses marching cubes on a voxel grid. The browser version uses a lighter stacked-outline mesh, so the two surfaces are similar and not identical.

11. A numerical finding: 16-bit ranking

This is the most useful technical finding of the project for a paper, because it can affect anyone who trains with mixed precision and deploys at full precision.

The symptom

The same weights scored about 1 mm on training sessions on the GPU and 4 to 6 mm on the laptop. The held-out box went from 3.6 mm under 16-bit autocast to 9.8 mm at 32-bit precision. The ONNX export also failed its accuracy check, as if the model were chaotic.

The cause

AdaPoinTr ranks its candidate query points with a final sigmoid score and keeps the top ones. Training ran under float16 autocast, where the score is rounded to 16 bits. Near 1 many scores become exactly equal, so the order follows the point index. The network was trained under that behaviour. At 32 bits the ties disappear, the scores order the points differently, and a different query set is selected.

Testing half precision on each network part in turn isolated it: only the query-ranking head changed the result.

The fix

```
query_ranking = self.query_ranking(coarse)
ties = torch.arange(n)[None, :, None]
query_ranking = query_ranking.half().float() - 2e-7 * ties
idx = torch.argsort(query_ranking, dim=1, descending=True)
```

Measure	Before	After
Held-out bottle at 32-bit, mm	6.82	4.79
Held-out box at 32-bit, mm	9.83	3.12
Median output height, bottle, cm	21.2	23.0
ONNX export probes passed	22 of 23	23 of 23
ONNX exact agreement	16 of 23	22 of 23

Table 15. Effect of the fix, with no retraining. The ONNX "before" row was measured on the previous checkpoint.

12. The web application

The application is a set of static files in the **webapp** folder. A plain local server only hands them over. The browser downloads the model and runs everything: file parsing, thinning, cleaning, the model through ONNX Runtime Web on WebAssembly, and the surface.

```
python3 -m http.server 8000 --directory webapp
```

View	Shows
1. Raw input	Every point in the file
2. Cleaned input	The object points the cleaner kept
3. Object point cloud	16,384 points from the model
4. 3D object	The closed surface

Table 16. The four views.

The model is 139 MB, split into three parts. On the development laptop one scan took about 4 seconds. JavaScript preprocessing matches Python bit for bit on 42 files. A real phone had not been tested at the time of writing; low-memory phones may be unable to load the model.

Native apps

The same files are packaged as two apps, so all four views behave the same. **Android:** DeepSonar3D-4.0.apk, 129 MB, with the model inside and no network needed. Tested on an Android 16 emulator with 2.5 GB of memory: a real scan took 6.6 seconds and gave all four views. **Linux:** DeepSonar3D-4.0.AppImage, 241 MB, an Electron desktop app. Tested with real bottle and box scans: about 4 to 7 seconds per scan.

13. Limits and threats to validity

- **Two objects, one room, one device.** All real data comes from one bottle and one box, in one room, on one device.
- **Instance-specific real targets.** The final model has seen the full form of this bottle and this box, so real-scan accuracy does not transfer to other objects.
- **Held-out sessions share the objects.** They test new scans, not new objects.
- **Reduced input sensitivity.** The final model fails both probe gates (Table 10). Narrowing the input hardly changes the output, so it will not faithfully follow a differently sized object.
- **Reference surfaces are not measurements.** Their registration error is 4 to 11 mm, which is similar to or larger than some reported distances. Distances below that level should not be read as true accuracy.
- **The box depth of 2 cm is an assumption.** Sizes were not measured with a ruler. The bottle outline came from a vision model reading a photo.
- **Two bottle sessions lack targets.** 123027/123028 and 123523 still output the smeared width.
- **Simulated test objects share room data with training.** The test covers new objects, not unseen rooms.
- **No upright, no output.** Objects lying flat are lost to the background.
- **Hidden sides are predicted.** Nothing here is a measurement of the back.

14. Project journey

Date	Step	Outcome
17 Oct	AdaPoinTr ported; first fine-tune and segmentation	Worked on paper; results were not trustworthy
20 Oct	Rebuild from the data up: forensics of files, a device-faithful simulator	Found how files are actually produced
17 Oct	An overfit template version was rejected by the owner and replaced	Rule set: the model must construct the object
28 Oct	Geometry-only cleaner	Object found without labels
30 Oct	Raw metric files arrived; simulator rebuilt to replay real sessions; dataset generated	66,000 pairs
30 Oct	Stage 1, a launch was first refused by the budget guard and unnoticed	Monitoring tightened; heights good, shapes fuzzy
31 Oct	Camera, hybrid and Pixal3D options reviewed; hybrid tried in a short trial	Not shipped
31 Oct	Real-scan stage; run died with a local client after 76 minutes	Launcher changed so that runs have no local process
1 Nov	Branch A compared with Branch B, the owner's full-target idea	Branch B clearly better on real bottles
1 Nov	Box targets added; plain 10-epoch final run	Final model
1 Nov	Surface step, browser application, 16-bit ranking found and fixed	Browser results match training

Table 17. Timeline.

Lessons

- Data faithfulness matters more than training length. More epochs of the wrong loss gave nothing.
- Validation on simulation did not show the real improvement. Real-scan views and held-out sessions did.
- Test the deployed precision, not only the training precision.
- Try new ideas in short, time-boxed trials with a pass rule written first.
- Remote jobs must not depend on a local process, and monitoring must cover every failure state.

15. Exhibition guide

1. Start the server on the laptop and connect phones to the same Wi-Fi.
2. Open the page. Wait for "Model ready on this device".
3. Load a held-out scan: bottle 123815 or box 121600.
4. Walk through the four views from left to right.

Questions to expect

Question	Answer
Is the back measured?	No. It is predicted. The sensor sees one side.
Does it work on any object?	No. It is tuned to this bottle and box. For a different object expect a result pulled toward what it learned.
Is it a template?	There is no label, lookup or stored shape in the model: the input is 2048 points. But it was fine-tuned on this bottle and box, and the width probe shows a strong pull toward them. Say so directly.
How do you know the box depth?	We do not, from the scan. 2 cm was assumed.
Does it run on a server?	No. The server only hands over the files. The model runs in the browser.
Why not use the camera?	It is a promising extension and was tried. It needs paired photos and scans.

Table 18. Likely questions.

16. Towards a paper

Possible title: One-view object completion from a low-resolution time-of-flight sensor, with session-replay simulation and known-object fine-tuning.

Possible contributions

- Session-replay simulation: real frames with a new object ray-cast in, for a sensor of 8 by 8 zones.
- A label-free cleaner for hand-held maps with untracked motion.
- Known-object registered targets for real-domain fine-tuning, and evidence of their effect compared with a front-only loss.
- A reproducible mixed-precision deployment failure in a ranking step, with a simple fix.

What a paper still needs

- Ruler measurements, and ideally a scanned ground-truth back for each object.
- More objects and rooms, with object-level hold-out.
- A baseline on the same data with a second network.
- Repeated runs with different seeds.
- A review of the mixed-precision behaviour on other query-based models.

17. Reproducibility

```
uv run python -m scripts.evaluate_final --ckpt checkpoints/adapointr_final.pth --final \
    --run-log logs/final_final7.jsonl --per-kind 0 --out outputs/final
uv run python -m scripts.rank_checkpoints outputs/final/checkpoint_ranking2.json final7=checkpoints/a
dapointr_final.pth
uv run python -m scripts.training_history
PYTHONPATH=. uv run --with pytest python -m pytest tests -q
node web/test/parity.mjs
```

Item	Location
Final weights	checkpoints/adapointr_final.pth
Browser model	webapp/model
Configuration	configs/final.yaml, configs/real_*_parts.json
Run logs and metrics	docs/evidence, logs, outputs/training_history
Evaluation	outputs/final/evaluation.md

Table 19. Key artefacts.

References

Yu, X., Rao, Y., Wang, Z., Lu, J., Zhou, J. AdaPoinTr: Diverse point cloud completion with adaptive geometry-aware transformers. IEEE TPAMI, 2023.

Yu, X., Rao, Y., Wang, Z., Liu, Z., Lu, J., Zhou, J. PoinTr: Diverse point cloud completion with geometry-aware transformers. ICCV, 2021.

Yuan, W., Khot, T., Held, D., Mertz, C., Hebert, M. PCN: Point completion network. 3DV, 2018.

STMicroelectronics. VL53L5CX time-of-flight 8 x 8 multizone ranging sensor, datasheet.

Official AdaPoinTr code: github.com/yuxumin/PoinTr.